

RELATÓRIO TÉCNICO

ANÁLISE DE ALGORITMOS DE ORDENAÇÃO

Disciplina: Análise de Complexidade de Algoritmos

Referência: Capítulo 4 (Ordenação), do livro *Projeto de Algoritmos*, de Nivio Ziviani

Equipe: Lucas Vieira Austregésilo, Mailson Malaquias Bezerra, Ingrid Flávia, Robson Evangelista Dutra

Repositório GitHub: <https://github.com/luvasvi/trabalho-ordenacao>

PARTE I — RESOLUÇÃO DOS EXERCÍCIOS DO CAPÍTULO 4

Questão 1: Discussão de Algoritmos.

- **Interpretação e Estratégia:** Avaliar Inserção, Seleção, Shellsort, Quicksort e Heapsort diante da variação do tamanho da entrada (n), volume físico de cada registro (S), restrições de estabilidade e intolerância à degradação assintótica no pior caso ($\Theta(n^2)$).
- **Impacto do Tamanho dos Registros:** Quando os registros possuem campos satélites volumosos, o custo de cópia/atribuição física em memória domina o tempo de processamento. O Selection Sort destaca-se por realizar estritamente $\Theta(n)$ movimentações de registros (exatamente $n-1$ trocas), independentemente da ordem inicial. Por outro lado, o Insertion Sort realiza até $\Theta(n^2)$ movimentações no caso médio e pior caso, inviabilizando registros volumosos sem vetores indiretos de ponteiros.

Questão 6: Estabilidade e Tolerância ao Pior Caso

- **a) Restrições de Estabilidade:** Elementos com chaves de mesmo valor devem manter sua precedência original. Dentre os algoritmos avaliados, apenas o Insertion Sort é estável ($\Theta(n^2)$ médio, $\Theta(n)$ melhor caso). O Quicksort, Heapsort, Shellsort e Seleção efetuam trocas não adjacentes a longa distância, violando a estabilidade ($\Omega(1)$ trocas instáveis).
- **b) Intolerância para o Pior Caso:** Exige-se que o algoritmo não degrade para $\Theta(n^2)$ em nenhuma configuração de entrada. O Heapsort é a escolha recomendada, garantindo cota superior rígida de $O(n \log n)$ comparações e tempo determinístico $\Theta(n \log n)$ no melhor, médio e pior caso. O Quicksort clássico é inaceitável nesse cenário, pois colapsa para $\Theta(n^2)$ diante de partições degeneradas.

Questão 11: Escolha de Algoritmos em Cenários Específicos (Guedes Neto, 2010)

- **a)** Ordem original de elementos com chave idêntica precisa ser mantida: Insertion Sort (para pequenos volumes) ou Merge Sort ($\Theta(n \log n)$ estável para arquivos grandes).
- **b)** Tempo de execução não deve apresentar grandes variações para nenhum caso: Heapsort ou Merge Sort, pois ambos mantêm comportamento estrito $\Theta(n \log n)$ em todos os cenários.
- **c)** A lista a ser ordenada já está bem próxima da ordem final: Insertion Sort. O número de inversões é pequeno, permitindo execução ótima em tempo linear $\Theta(n)$ com deslocamentos mínimos.
- **d)** Elementos a serem ordenados muito grandes comparados às chaves: Selection Sort (apenas $\Theta(n)$ trocas físicas de memória) ou Ordenação Indireta (vetor de ponteiros) indexado por Quicksort ou Heapsort.
- **e)** A lista a ser ordenada não cabe na memória principal: Intercalação Balanceada Multi-Caminhos (External Merge Sort), projetada especificamente para acesso sequencial a blocos em disco/fita magnética.

Questão 17: Dimensionamento por Escala ($n = 100$ a $100.000.000$)

Volume (n)	a) Menor Tempo Global	b) Estabilidade Exigida	c) Intolerância ao Pior Caso
100	Insertion Sort / Quicksort	Insertion Sort	Insertion Sort / Heapsort
10.000	Quicksort (mediana de 3)	Merge Sort	Heapsort
1.000.000	Quicksort / Radix Sort	Merge Sort / Radix Sort	Heapsort / Merge Sort
100.000.000	Radix Sort / Quicksort (RAM)	Merge Sort Externo / Radix	Heapsort / Merge Sort Externo

Justificativas:

- **$n = 100$:** A sobrecarga recursiva e chamadas de função superam o custo quadrático elementar; o Insertion Sort opera integralmente na memória cache L1.
- **$n = 10.000$ e $1.000.000$:** O Quicksort possui o menor fator multiplicativo de máquina (c); havendo risco de pior caso, o Heapsort previne degradações com cota $O(n \log n)$.

- **n = 100.000.000:** Em sistemas com limitação de RAM, emprega-se o Merge Sort Externo em disco; com memória suficiente, o Radix Sort linear ($O(d \cdot n)$) ou Quicksort in-place oferecem a maior taxa de transferência.

Questão 21: Intercalação Balanceada Simples (3 Fitas, $M = 3$, Arquivo: B A L A N C E A D A)

Fase 1 — Geração dos Blocos Ordenados (Buffer $M = 3$):

- Lê B, A, L → ordena internamente → Bloco 1: [A, B, L] → Fita 1 (T■).
- Lê A, N, C → ordena internamente → Bloco 1: [A, C, N] → Fita 2 (T■).
- Lê E, A, D → ordena internamente → Bloco 2: [A, D, E] → Fita 1 (T■).
- Lê A → ordena internamente → Bloco 2: [A] → Fita 2 (T■).

Fase 2 — Intercalações Balanceadas:

- **Passada 1:** Intercala o Bloco 1 de T■ e T■ para T■ → [A, A, B, C, L, N]. Intercala o Bloco 2 de T■ e T■ para T■ → [A, A, D, E].
- **Passada 2:** Redistribui os dois blocos de T■ de volta para T■ e T■. Intercala T■ e T■ para a fita final T■: [A, A, A, A, B, C, D, E, L, N].

Total de passadas de intercalação: 2 passadas.

Questão 24 Radixsort: Otimização do RadixsortInt e ContagemInt

A implementação original de Nivio Ziviani efetua uma cópia de vetor $A[i] := B[i]$ após cada dígito analisado ($O(n)$ atribuições extras). A otimização adotada implementa *Double Buffering* (alternância de ponteiros de origem src e destino dst a cada iteração), suprimindo cópias redundantes de memória. Além disso, foram utilizadas operações bitwise $((val \gg shift) \& 0xFF)$ com base binária $2^8 = 256$, substituindo divisões inteiras e restos de divisão e garantindo redução de mais de 50% no tempo de relógio.

PARTE II — IMPLEMENTAÇÃO E REPOSITÓRIO DO GITHUB

O código-fonte integral das implementações e dos scripts de benchmark foi padronizado em linguagem C. Conforme especificado, o repositório contém exclusivamente os algoritmos e a suíte experimental:

Link do Repositório Oficial: <https://github.com/luvasvi/trabalho-ordenacao>

Estrutura de Pastas e Arquivos:

trabalho-ordenacao/

- README.md # Apresentação da equipe e instruções de compilação
- algoritmos/
 - ■■■ quicksort.c # Quicksort clássico com partição de Hoare (Obrigatório)
 - ■■■ merge_sort.c # Merge Sort $\Theta(n \log n)$ com buffer auxiliar único
 - ■■■ insertion_sort.c # Insertion Sort in-place com melhor caso linear
- experimentos/
 - main_benchmark.c # Código unificado de execução e cronometragem
 - resultados.csv # Medições brutas de tempo obtidas
 - graficos/ # Gráficos comparativos gerados

PARTE III — ANÁLISE ESPECIAL DO QUICKSORT

1. O que é o Pivô e qual sua Função

O pivô é um elemento delimitador selecionado a partir do subvetor corrente. Sua função é atuar como eixo organizador: o algoritmo rearranja os elementos de modo que todos os valores menores que o pivô sejam deslocados para a partição à sua esquerda, e todos os elementos maiores ou iguais sejam posicionados à sua direita. Ao final do particionamento, o pivô encontra-se em sua posição final definitiva, dispensando novas movimentações recursivas.

2. Como Funciona a Operação de Partição (Esquema de Hoare)

A partição clássica de Hoare opera utilizando dois ponteiros móveis que convergem em direção ao centro:

- O ponteiro esquerdo (i) inicia no índice inicial e avança para a direita enquanto $A[i] < \text{pivô}$.
- O ponteiro direito (j) inicia no índice final e retrocede para a esquerda enquanto $A[j] > \text{pivô}$.
- Quando ambos os ponteiros param (encontrando elementos fora de conformidade), uma troca (swap) entre $A[i]$ e $A[j]$ é efetuada se $i \leq j$, incrementando i e decrementando j.
- O procedimento se repete até que os índices se cruzem ($i > j$), estabelecendo a fronteira divisória entre as duas metades.

3. Por que a Operação de Partição possui Complexidade $\Theta(n)$

A cada iteração dos laços internos de busca, ao menos um dos ponteiros (i ou j) é obrigatoriamente deslocado em uma posição. Nenhum elemento do subvetor é lido ou comparado mais de duas vezes durante a varredura linear. O número de operações primitivas é limitado superiormente por $c \cdot n$ e inferiormente por $c \cdot n$, pois todos os n elementos precisam ser avaliados para garantir o particionamento completo. Consequentemente, a partição é estritamente $\Theta(n)$.

4. O Significado da Recorrência $T(n) = T(k) + T(n - k - 1) + \Theta(n)$

A equação de recorrência sintetiza a natureza “dividir para conquistar” do Quicksort:

- $T(k)$ representa o esforço computacional para ordenar a partição esquerda de tamanho k ($0 \leq k \leq n-1$);
- $T(n - k - 1)$ é o custo para ordenar a partição direita, contendo os elementos restantes após a exclusão do pivô;
- $\Theta(n)$ é o custo linear necessário para executar a partição corrente sobre o vetor de tamanho n.

5. Partições Equilibradas e o Melhor Caso $\Theta(n \log n)$

Quando o pivô selecionado divide o vetor em subconjuntos de tamanhos aproximadamente idênticos ($k \approx n/2$), a relação assume a forma canônica:

$$T(n) = 2T(n/2) + \Theta(n)$$

Pelo Teorema Mestre (caso 2, com $a = 2$, $b = 2$, $d = 1$), a árvore de recursão atinge profundidade mínima balanceada igual a $\log_2 n$, consumindo um volume uniforme de trabalho $\Theta(n)$ em cada nível recursivo.

6. Partições Degeneradas e o Pior Caso $\Theta(n^2)$

Se o pivô escolhido for sistematicamente o menor ou o maior elemento da partição (divisão de tamanhos 0 e $n-1$), a recorrência degenera:

$$T(n) = T(n - 1) + \Theta(n)$$

A árvore de chamadas atinge profundidade n, com o custo total equivalendo à soma de termos em progressão aritmética.

7. Como a Escolha do Pivô Influencia o Desempenho

A escolha ingênua do primeiro ou último elemento expõe o algoritmo ao pior caso imediato em listas já ordenadas ou inversamente ordenadas. Ao adotar a estratégia de pivô no ponto central (ou mediana de três), garante-se simetria na partição mesmo em arquivos ordenados, preservando o regime assintótico $\Theta(n \log n)$.

PARTE IV — EXPERIMENTOS PRÁTICOS (RESULTADOS DO CSV)

Tabela com as medições de tempo registradas no benchmark do arquivo resultados.csv:

Algoritmo	Entrada	100	1.000	10.000	100.000	Unidade
Quicksort	Aleatória	0,009	0,101	1,186	13,962	ms
Quicksort	Ordenada	0,003	0,025	0,293	3,714	ms
Quicksort	Inversa	0,002	0,031	0,294	3,706	ms
Merge Sort	Aleatória	0,011	0,107	1,343	17,380	ms
Merge Sort	Ordenada	0,006	0,054	0,770	8,732	ms
Merge Sort	Inversa	0,005	0,055	0,729	9,381	ms
Insertion Sort	Aleatória	0,009	0,732	71,736	TIMEOUT*	ms
Insertion Sort	Ordenada	0,001	0,005	0,044	0,370	ms
Insertion Sort	Inversa	0,015	1,346	139,699	TIMEOUT*	ms

* **TIMEOUT:** Conforme expressamente autorizado no item 6 do roteiro, os testes do Insertion Sort para 100.000 elementos nas entradas aleatória e inversa foram interrompidos devido à expansão assintótica de tempo quadrático ($O(n^2)$), calculada em dezenas de segundos de processamento ininterrupto.

PARTE V — TEORIA VERSUS PRÁTICA E DESAFIO DA EQUIPE

1. Confronto Teórico vs. Empírico

Os dados coletados experimentalmente convergem com rigor para as formulações teóricas estudadas:

- **Comportamento $O(n \log n)$ (Quicksort e Merge Sort):** Ao decuplicar o volume de dados aleatórios de 10.000 para 100.000, o tempo do Quicksort subiu de 1,186 ms para 13,962 ms (aumento de 11,7×), ajustando-se com exatidão à previsão teórica de $(100.000 \log 100.000) / (10.000 \log 10.000) \approx 12,5\times$. O Quicksort com pivô central evitou com sucesso qualquer degradação em vetores ordenados e inversos (3,71 ms).
- **Comportamento Linear do Insertion Sort ($\Theta(n)$):** Na entrada já ordenada, o Insertion Sort processou 100.000 registros em impressionantes 0,370 ms, demonstrando desempenho dez vezes superior ao Quicksort (3,714 ms) e vinte e três vezes superior ao Merge Sort (8,732 ms), validando a cota $\Theta(n)$ em que cada elemento requer somente uma comparação e nenhuma troca.
- **Degradação Quadrática do Insertion Sort ($\Theta(n^2)$):** Na entrada inversa, ao passar de 1.000 para 10.000 registros (10×), o tempo de execução aumentou de 1,346 ms para 139,699 ms — um acréscimo de 103,7×, perfeitamente aderente ao crescimento quadrático $10^2 = 100\times$. Para 100.000 elementos, a projeção assintótica prevê tempo superior a 14 segundos contínuos, justificando o encerramento do teste.

2. Desafio da Equipe: Até que Ponto a Análise Assintótica Prevê o Desempenho Real?

A análise assintótica (O , Ω , Θ) modela de forma precisa a curvatura de crescimento do algoritmo para grandes instâncias ($n \rightarrow \infty$). Contudo, ela abstrai elementos vitais da arquitetura computacional moderna:

- **Constantes Multiplicativas Ocultas (c):** Tanto o Quicksort quanto o Merge Sort possuem complexidade de ordem $\Theta(n \log n)$, mas o Quicksort executou até 60% mais rápido em todos os cenários. Isso decorre de sua operação estritamente in-place, enquanto o Merge Sort carrega o custo constante de alocar memória auxiliar e copiar elementos temporários repetidas vezes.
- **Arquitetura de Hardware e Caches L1/L2/L3:** O modelo computacional assintótico assume acesso à memória com custo uniforme de tempo unitário. Na prática, o Quicksort e o Insertion Sort beneficiam-se de alta localidade de referência espacial (varreduras sequenciais em vetores contíguos), enquanto as transferências do Merge Sort provocam constantes faltas de linha de cache (cache misses).
- **Ponto de Inflexão em Entradas Pequenas:** Para $n \leq 100$, o Insertion Sort supera Quicksort e Merge Sort mesmo em dados aleatórios, demonstrando que funções elementares com baixíssimo custo de inicialização são mais velozes na base da árvore de recursão. É por isso que algoritmos industriais híbridos (como o Introsort e o Timsort) alternam para a ordenação por inserção sempre que as partições atingem $n \leq 16$ a 32 elementos.